

Influenza Deaths Forecast

Anurag Nepal

2026-10-04

Background

I wanted to forecast US pneumonia and influenza mortality as a time series problem. Weekly surveillance death counts have strong seasonality and ugly reporting artifacts, including the COVID-era spikes. The question was which modeling approach gives a usable 24-month forecast: ETS, ARIMA on raw versus cleaned data, and finally a hybrid that lets volatility vary over time.

Setup

The analysis reads `National_Custom_Data.csv`, which is not in this repo. It is a custom data download from CDC FluView's pneumonia, influenza, and COVID-19 mortality surveillance (NCHS), National region. To reproduce: open the FluView mortality surveillance page, use the Downloads tool, and export the custom National-level dataset. The columns used here are `SEASON`, `WEEK`, `NUM.INFLUENZA.DEATHS`, `NUM.PNEUMONIA.DEATHS`, and `TOTAL.DEATHS`.

```
# Load necessary libraries
library(forecast)
library(tseries)
library(dplyr)
library(lubridate)
library(knitr)
library(ggplot2)
library(astsa)
library(itsmr)
library(ltsa)
library(car)
```

Method

Loading and cleaning

The raw counts arrive comma-formatted, so the first job is parsing them into numbers. Flu seasons run week 40 to week 39, so I map each season/week pair to a calendar date, then aggregate to monthly level. The target is the percent of all deaths attributable to pneumonia and influenza. I keep two versions of the series: the raw one, and a cleaned one with outliers interpolated via `tsclean` for more robust modeling.

```
# =====
```

```
# DATA LOADING & CLEANING
```

```
# =====
```

```
# Load Data
```

```
# National_Custom_Data.csv holds weekly influenza/pneumonia death counts from
```

```
# CDC FluView (NCHS mortality surveillance), National region custom download.
```

```
# It is not in this repo; place it next to this file to re-run the analysis.
```

```
raw_data <- read.csv("National_Custom_Data.csv", stringsAsFactors = FALSE)
```

```
head(raw_data)
```

```
##      AREA SUB.AREA AGE.GROUP SEASON WEEK THRESHOLD BASELINE PERCENT.I
## 1 National      NA    18-64  2025-26   40      5.0      4.5      NA
## 2 National      NA     65+  2025-26   40      6.3      5.8      NA
## 3 National      NA    18-64  2025-26   41      5.1      4.6      NA
## 4 National      NA     65+  2025-26   41      6.4      5.9      NA
## 5 National      NA    18-64  2025-26   42      5.2      4.7      NA
## 6 National      NA     65+  2025-26   42      6.5      6.0      NA
##  PERCENT.P.I PERCENT.PIC NUM.INFLUENZA.DEATHS NUM.PNEUMONIA.DEATHS
## 1          4.2          4.4                1                485
## 2          5.6          6.0               16               2,343
## 3          4.3          4.4                3                495
## 4          5.7          6.1               18               2,422
## 5          4.6          4.7                2                527
## 6          5.8          6.1               16               2,481
##  NUM.COVID.19.DEATHS TOTAL.PIC TOTAL.DEATHS PERCENT.COMPLETE
## 1                 39        505       11,534             > 100%
## 2                367       2,551       42,181             > 100%
## 3                 34        517       11,712             > 100%
## 4                290       2,619       42,637             > 100%
## 5                 27        540       11,508             > 100%
## 6                248       2,636       43,419             > 100%
```

```
# Clean Numerical Columns (Remove commas)
```

```
clean_col <- function(x) as.numeric(gsub(",", "", x))
```

```
raw_data$NUM.INFLUENZA.DEATHS <- clean_col(raw_data$NUM.INFLUENZA.DEATHS)
```

```
raw_data$NUM.PNEUMONIA.DEATHS <- clean_col(raw_data$NUM.PNEUMONIA.DEATHS)
```

```
raw_data$TOTAL.DEATHS <- clean_col(raw_data$TOTAL.DEATHS)
```

```
# Create Date Column
```

```
get_cal_year <- function(season, week) {
```

```
  start_year <- as.numeric(substr(season, 1, 4))
```

```
  if (week >= 40) return(start_year)
```

```
  else return(start_year + 1)
```

```
}
```

```
raw_data$Year <- mapply(get_cal_year, raw_data$SEASON, raw_data$WEEK)
```

```
raw_data$Date <- as.Date(paste(raw_data$Year, raw_data$WEEK, 1, sep="-"), format="%Y-%U-%u")
```

```
# Aggregate to Monthly Level
```

```
monthly_data <- raw_data %>%
```

```
  group_by(MonthDate = floor_date(Date, "month")) %>%
```

```
  summarise(
```

```
    Total_P_I = sum(NUM.INFLUENZA.DEATHS + NUM.PNEUMONIA.DEATHS, na.rm = TRUE),
```

```
    Total_All_Deaths = sum(TOTAL.DEATHS, na.rm = TRUE)
```

```

) %>%
mutate(Percent_PI = (Total_P_I / Total_All_Deaths) * 100) %>%
arrange(MonthDate) %>%
filter(!is.na(MonthDate))

# Create Time Series Objects
start_yr <- year(min(monthly_data$MonthDate))
start_mo <- month(min(monthly_data$MonthDate))

# 1. Raw Data
ts_raw <- ts(monthly_data$Percent_PI, start = c(start_yr, start_mo), frequency = 12)

# 2. Cleaned Data (Outliers Interpolated for Robust Modeling)
ts_cleaned <- tsclean(ts_raw)

```

```
summary(monthly_data$Percent_PI)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   5.150   5.768   6.437   7.687   8.395  18.968
```

```
summary(ts_cleaned)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   5.150   5.768   6.458   7.511   8.493  14.799
```

Exploring the series

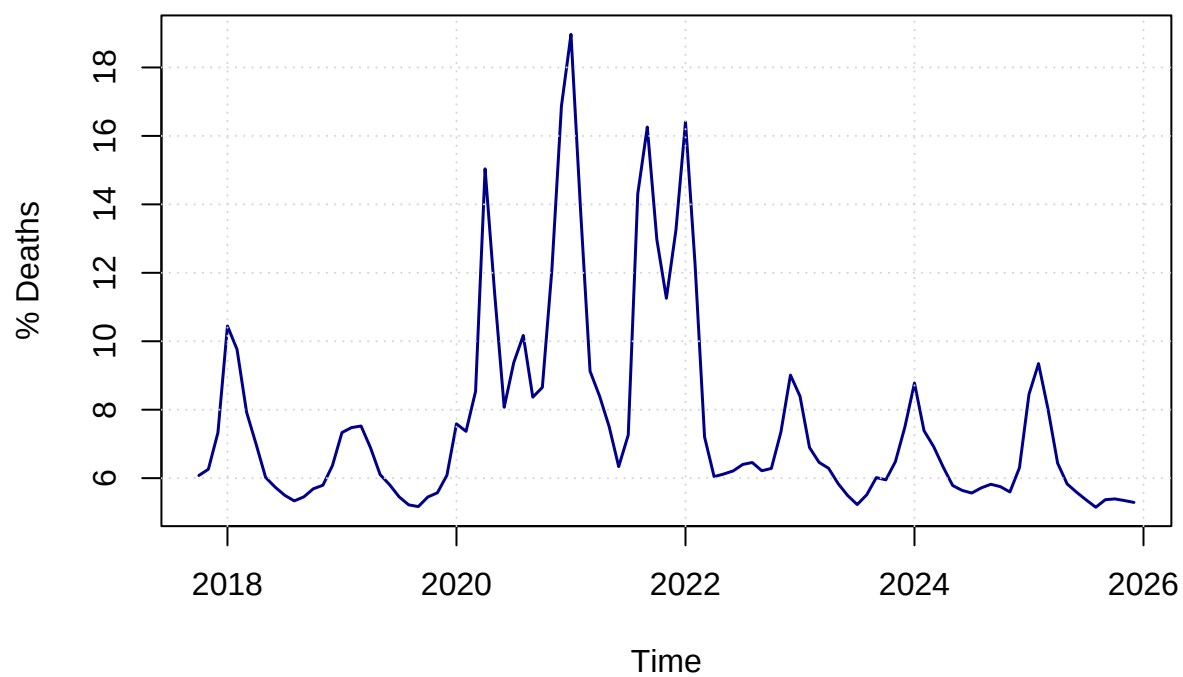
The raw plot shows the seasonal winter peaks plus the COVID-era spikes. After cleaning, the STL decomposition shows the seasonality clearly, and the ACF/PACF guide the ARIMA orders.

```

# Plot Raw Data
plot(ts_raw, main="US P&I Mortality Rate (Raw Data)", ylab="% Deaths", col="darkblue", lwd=1.5)
grid()

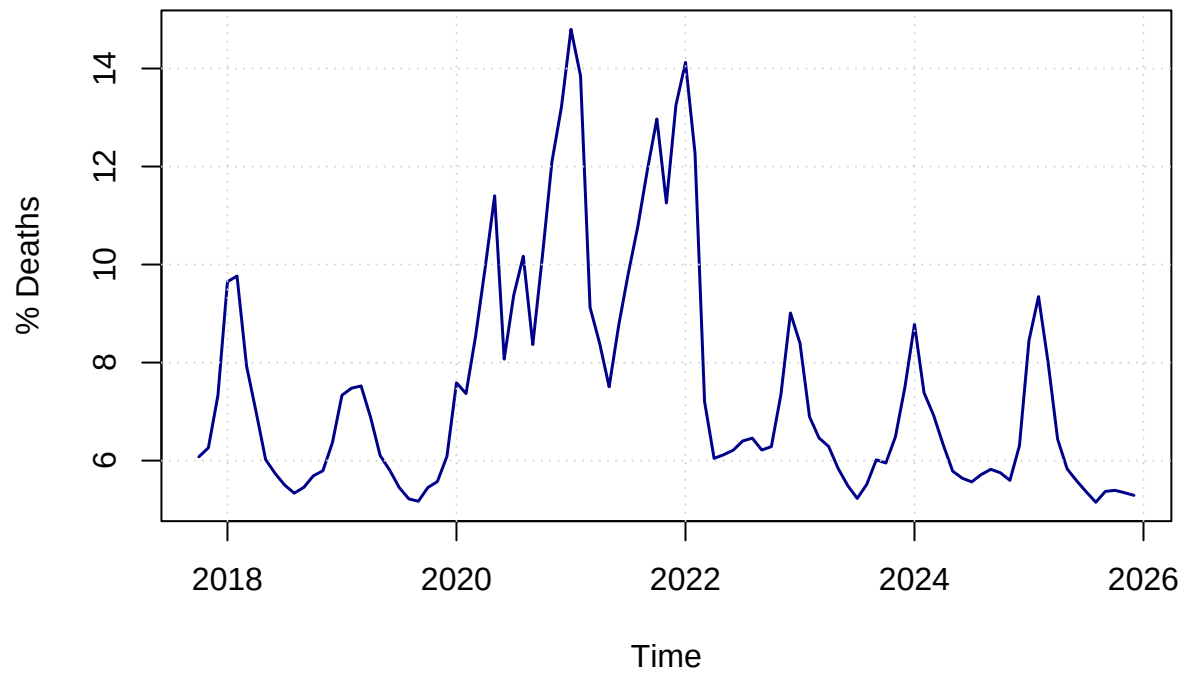
```

US P&I Mortality Rate (Raw Data)

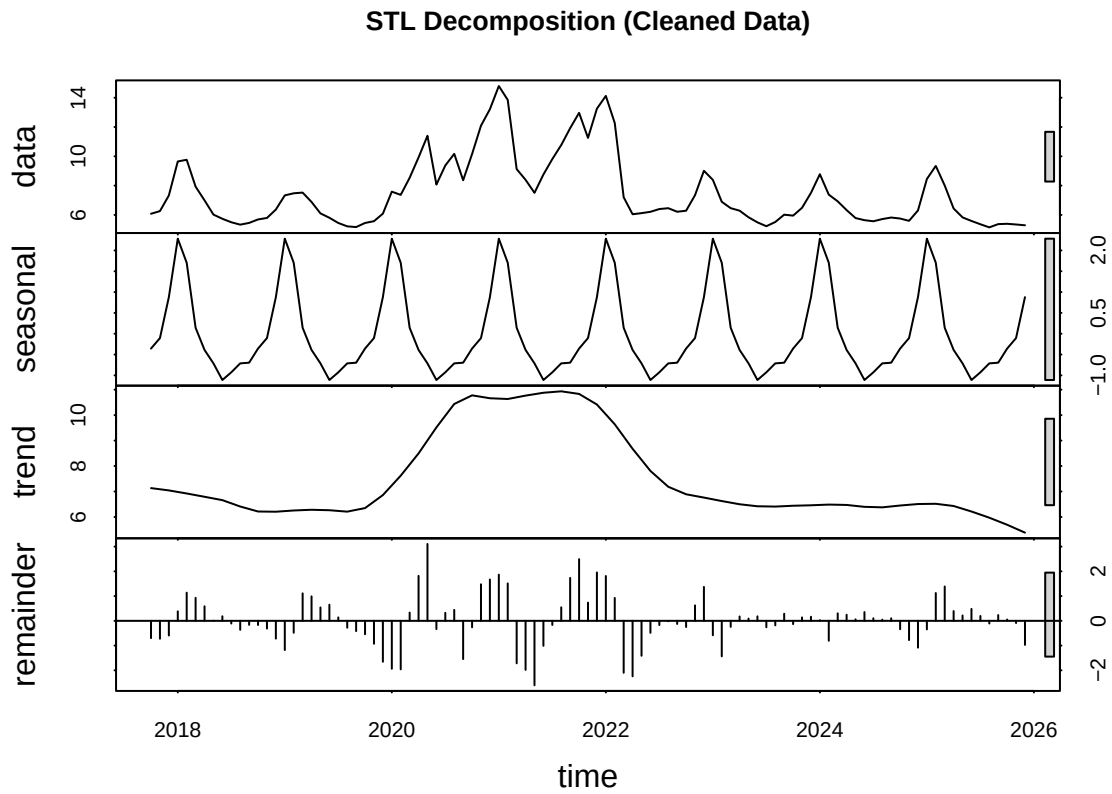


```
plot(ts_cleaned, main="US P&I Mortality Rate (Cleaned Data)", ylab="% Deaths", col="darkblue", lwd=1.5)  
grid()
```

US P&I Mortality Rate (Cleaned Data)

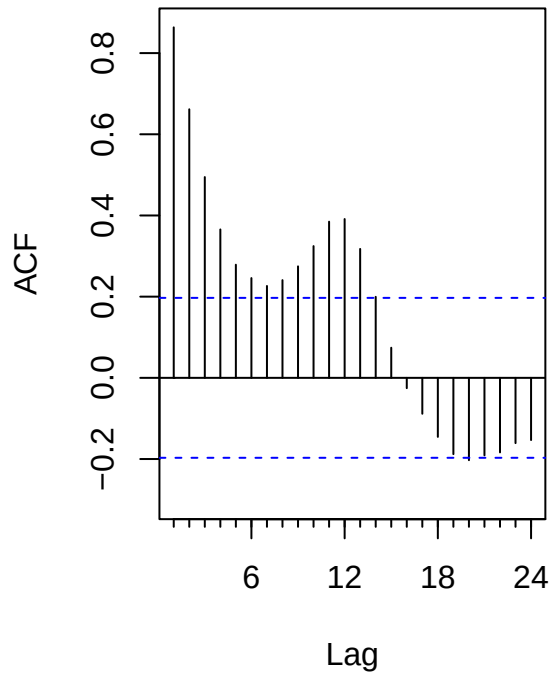


```
# STL Decomposition (Using Cleaned Data to see clear seasonality)
decomp <- stl(ts_cleaned, s.window="periodic")
plot(decomp, main="STL Decomposition (Cleaned Data)")
```

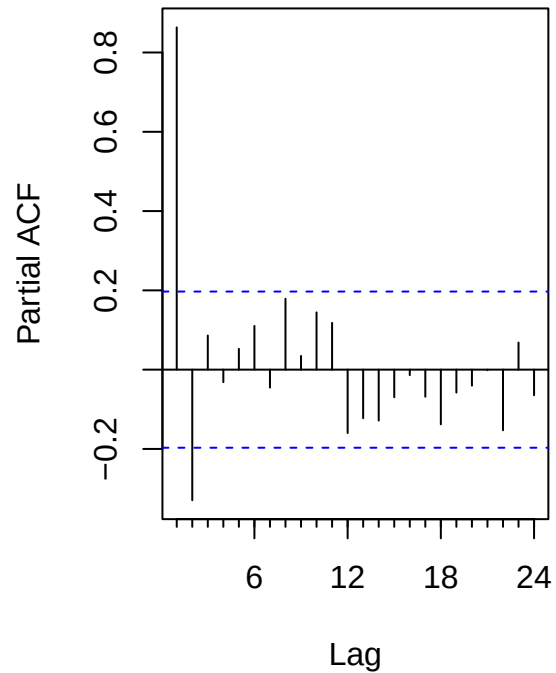


```
# ACF and PACF
par(mfrow=c(1,2))
Acf(ts_cleaned, main="ACF (Cleaned Data)")
Pacf(ts_cleaned, main="PACF (Cleaned Data)")
```

ACF (Cleaned Data)



PACF (Cleaned Data)



```
par(mfrow=c(1,1))
```

ETS baseline

First model: ETS with automatic error/trend/seasonality selection, fit on the cleaned series, with a 24-month forecast and residual diagnostics.

```
# We use ETS (Error, Trend, Seasonality).
# model="ZZZ" allows the function to automatically select the best error/trend/season type

# Fit ETS Model
fit_ets <- forecast::ets(ts_cleaned, model="ZZZ")

# Summary of chosen model
print(summary(fit_ets))

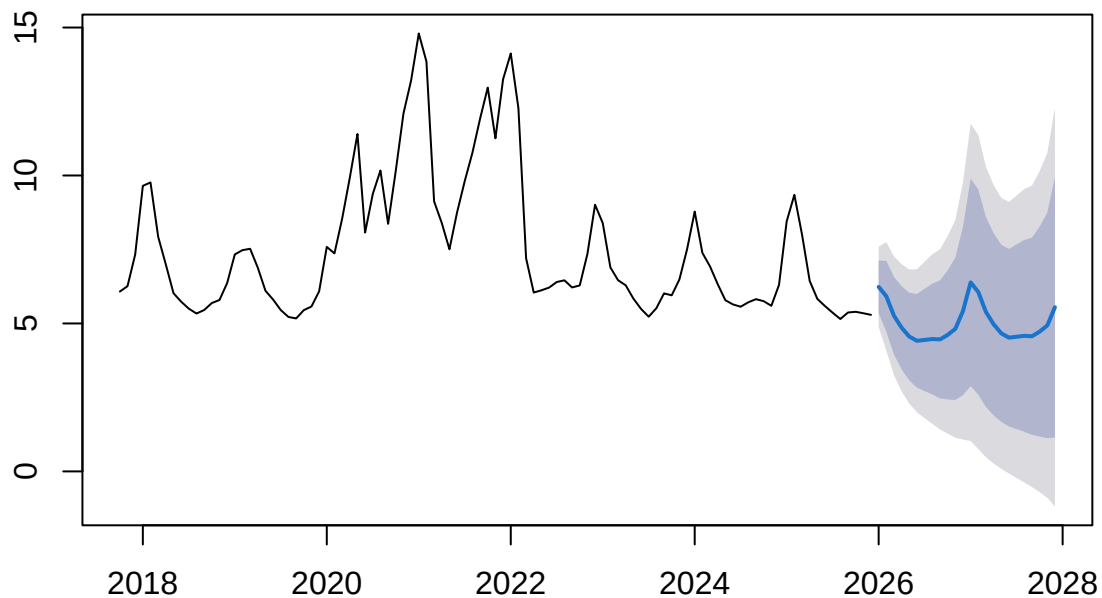
## ETS(M,A,M)
##
## Call:
## forecast::ets(y = ts_cleaned, model = "ZZZ")
##
## Smoothing parameters:
##   alpha = 0.9999
##   beta  = 0.0069
```

```
##      gamma = 1e-04
##
## Initial states:
##      l = 6.7712
##      b = 0.0502
##      s = 0.8951 0.8997 0.8954 0.8911 0.9217 0.9839
##          1.0683 1.2046 1.2719 1.0813 0.9631 0.9239
##
##      sigma: 0.1111
##
##      AIC      AICc      BIC
## 428.9658 436.5213 473.0828
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set -0.07002914 0.9571876 0.6270374 -1.419412 7.662248 0.3705041
##              ACF1
## Training set 0.1141287
```

```
# Plot ETS Forecast
ets_fc <- forecast::forecast(fit_ets, h=24)

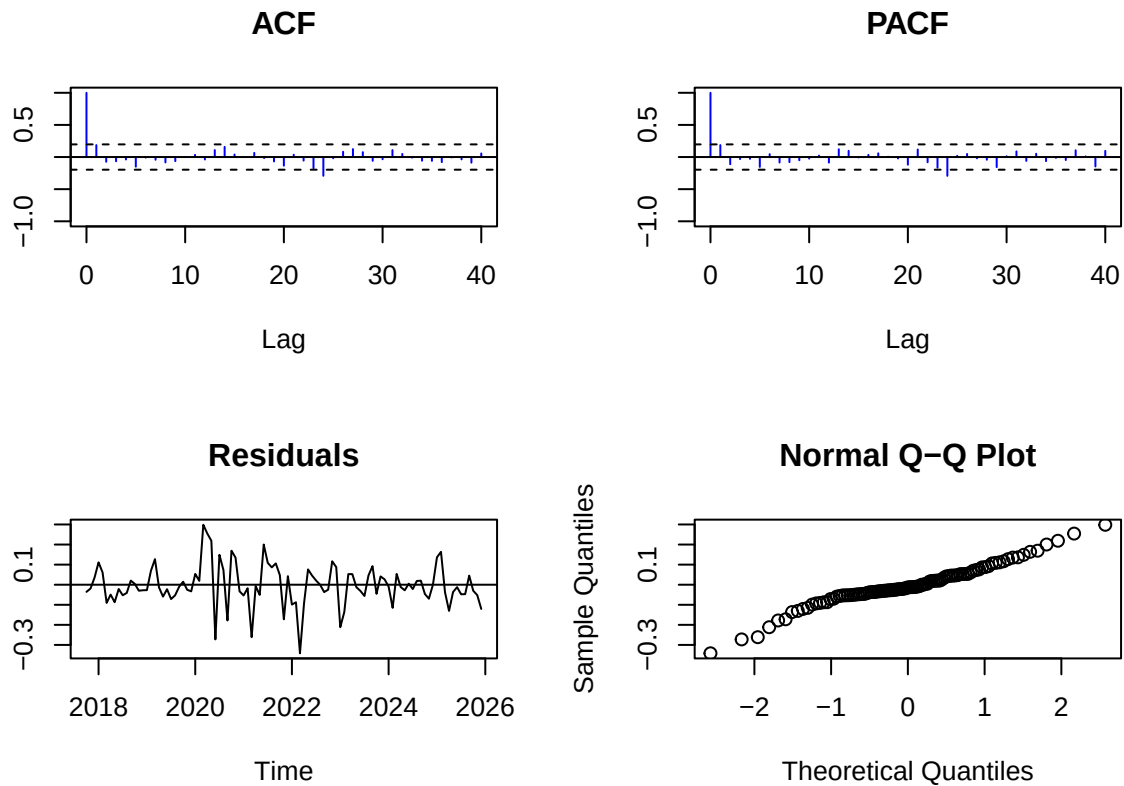
plot(ets_fc)
```

Forecasts from ETS(M,A,M)



```
test(ets_fc$residuals)
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)    16.76   0.6683
## McLeod-Li Q    Q ~ chisq(20)    30.69   0.0594
## Turning points T (T-64.7)/4.2 ~ N(0,1)    55     0.02 *
## Diff signs S    (S-49)/2.9 ~ N(0,1)    45     0.1659
## Rank P          (P-2425.5)/165.4 ~ N(0,1) 2312   0.4926
```



Checking stationarity

Before ARIMA, I ran ADF tests on the level, the seasonally differenced series, and the doubly differenced series to see how much differencing the data needs.

```
# 1. Test on Level Data
print("--- ADF Test: Level ---")
```

```
## [1] "--- ADF Test: Level ---"
```

```
print(aTSA::adf.test(ts_cleaned))
```

```

## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag      ADF p.value
## [1,]  0 -0.803  0.391
## [2,]  1 -1.026  0.310
## [3,]  2 -0.926  0.346
## [4,]  3 -1.067  0.295
## Type 2: with drift no trend
##      lag      ADF p.value
## [1,]  0 -2.54  0.1188
## [2,]  1 -3.50  0.0103
## [3,]  2 -2.91  0.0495
## [4,]  3 -2.87  0.0555
## Type 3: with drift and trend
##      lag      ADF p.value
## [1,]  0 -2.66  0.3015
## [2,]  1 -3.61  0.0361
## [3,]  2 -3.00  0.1607
## [4,]  3 -2.91  0.1998
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag      ADF      p.value
## [1,]  0 -0.8034741 0.3905352
## [2,]  1 -1.0262195 0.3102956
## [3,]  2 -0.9258934 0.3464361
## [4,]  3 -1.0674007 0.2954608
##
## $type2
##      lag      ADF      p.value
## [1,]  0 -2.541870 0.11883178
## [2,]  1 -3.497098 0.01029448
## [3,]  2 -2.906392 0.04952136
## [4,]  3 -2.867271 0.05545830
##
## $type3
##      lag      ADF      p.value
## [1,]  0 -2.656604 0.30154692
## [2,]  1 -3.607885 0.03612138
## [3,]  2 -3.002233 0.16070374
## [4,]  3 -2.906207 0.19983410

```

```

# 2. Test on Seasonal Difference (D=1)
ts_diff <- diff(ts_cleaned, lag=12)
print("--- ADF Test: Seasonal Difference (D=1) ---")

```

```

## [1] "--- ADF Test: Seasonal Difference (D=1) ---"

```

```

print(aTSA::adf.test(ts_diff))

```

```

## Augmented Dickey-Fuller Test

```

```

## alternative: stationary
##
## Type 1: no drift no trend
##      lag    ADF p.value
## [1,]  0 -2.66  0.0100
## [2,]  1 -2.93  0.0100
## [3,]  2 -2.50  0.0143
## [4,]  3 -2.54  0.0125
## Type 2: with drift no trend
##      lag    ADF p.value
## [1,]  0 -2.65  0.0914
## [2,]  1 -2.91  0.0499
## [3,]  2 -2.49  0.1409
## [4,]  3 -2.53  0.1255
## Type 3: with drift and trend
##      lag    ADF p.value
## [1,]  0 -2.72  0.275
## [2,]  1 -3.00  0.162
## [3,]  2 -2.60  0.324
## [4,]  3 -2.71  0.280
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag      ADF    p.value
## [1,]  0 -2.660852 0.01000000
## [2,]  1 -2.925013 0.01000000
## [3,]  2 -2.501546 0.01430213
## [4,]  3 -2.544426 0.01252926
##
## $type2
##      lag      ADF    p.value
## [1,]  0 -2.647268 0.09137071
## [2,]  1 -2.909390 0.04991106
## [3,]  2 -2.486887 0.14090899
## [4,]  3 -2.526822 0.12548383
##
## $type3
##      lag      ADF    p.value
## [1,]  0 -2.724249 0.2752020
## [2,]  1 -3.004827 0.1619747
## [3,]  2 -2.604446 0.3235489
## [4,]  3 -2.712112 0.2801000

# 3. Test on Double Difference (D=1, d=1)
# Often required if seasonal differencing alone doesn't remove the trend
ts_doubly_diff <- diff(ts_diff, lag=1)
print("--- ADF Test: Seasonal + First Difference (D=1, d=1) ---")

## [1] "--- ADF Test: Seasonal + First Difference (D=1, d=1) ---"

print(aTSA::adf.test(ts_doubly_diff))

## Augmented Dickey-Fuller Test

```

```
## alternative: stationary
##
## Type 1: no drift no trend
##      lag    ADF p.value
## [1,]  0 -8.65   0.01
## [2,]  1 -7.48   0.01
## [3,]  2 -5.90   0.01
## [4,]  3 -5.33   0.01
## Type 2: with drift no trend
##      lag    ADF p.value
## [1,]  0 -8.59   0.01
## [2,]  1 -7.43   0.01
## [3,]  2 -5.86   0.01
## [4,]  3 -5.30   0.01
## Type 3: with drift and trend
##      lag    ADF p.value
## [1,]  0 -8.54   0.01
## [2,]  1 -7.39   0.01
## [3,]  2 -5.84   0.01
## [4,]  3 -5.28   0.01
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
## $type1
##      lag    ADF p.value
## [1,]  0 -8.645996   0.01
## [2,]  1 -7.476852   0.01
## [3,]  2 -5.899900   0.01
## [4,]  3 -5.331257   0.01
##
## $type2
##      lag    ADF p.value
## [1,]  0 -8.594412   0.01
## [2,]  1 -7.431110   0.01
## [3,]  2 -5.863559   0.01
## [4,]  3 -5.297977   0.01
##
## $type3
##      lag    ADF p.value
## [1,]  0 -8.544893   0.01
## [2,]  1 -7.390294   0.01
## [3,]  2 -5.840880   0.01
## [4,]  3 -5.283964   0.01
```

ARIMA bake-off

Three candidates, all fit with `auto.arima`: one on the raw data, one on the cleaned data, and one on the cleaned data with a log transform. The trace output is hidden to keep the document readable.

```
# Model 1: ARIMA on Raw Data
fit_raw <- auto.arima(ts_raw, seasonal=TRUE, stepwise=FALSE, approximation=FALSE, trace=TRUE)

# Model 2: ARIMA on Cleaned Data (No Transformation)
fit_clean <- auto.arima(ts_cleaned, seasonal=TRUE, stepwise=FALSE, approximation=FALSE, trace=TRUE)
```

```

# Model 3: ARIMA on Cleaned Data + Log Transformation
fit_log_clean <- auto.arima(ts_cleaned, seasonal=TRUE, lambda=0, approximation=FALSE, trace=TRUE)

# Function to extract metrics
get_accuracy <- function(model, name) {
  acc <- accuracy(model)
  return(data.frame(
    Model = name,
    AICc = round(model$aicc, 2),
    RMSE = round(acc[2], 4),
    MAPE = round(acc[5], 4)
  ))
}

# Combine into a table
comparison_table <- rbind(
  get_accuracy(fit_raw, "ARIMA (Raw Data)"),
  get_accuracy(fit_clean, "ARIMA (Cleaned Data)"),
  get_accuracy(fit_log_clean, "ARIMA (Cleaned + Log)")
)

kable(comparison_table, caption = "Model Accuracy Comparison")

```

Table 1: Model Accuracy Comparison

Model	AICc	RMSE	MAPE
ARIMA (Raw Data)	363.24	1.4169	9.9286
ARIMA (Cleaned Data)	304.87	1.0744	9.1604
ARIMA (Cleaned + Log)	-127.60	1.1057	8.6107

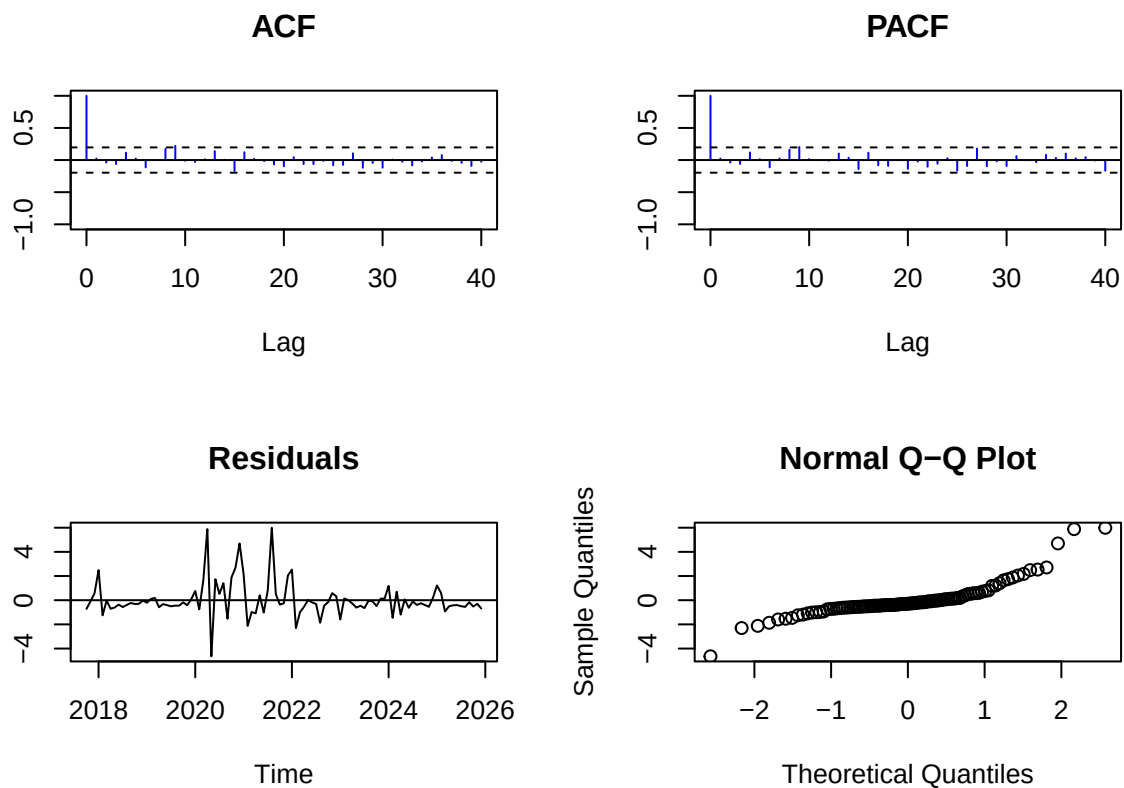
Residual diagnostics for all three fits:

```
test(fit_raw$residuals)
```

```

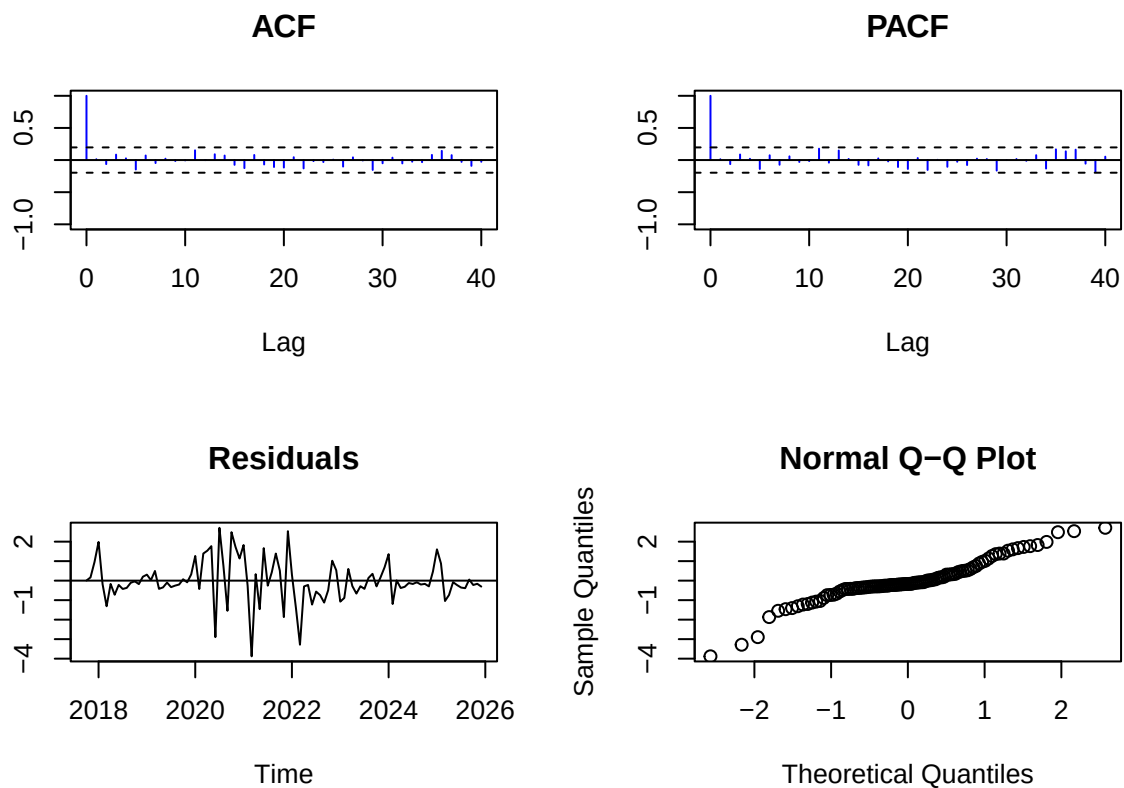
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)    21.66   0.3593
## McLeod-Li Q    Q ~ chisq(20)    47.23   5e-04 *
## Turning points T (T-64.7)/4.2 ~ N(0,1)    63    0.6884
## Diff signs S    (S-49)/2.9 ~ N(0,1)    54    0.0833
## Rank P          (P-2425.5)/165.4 ~ N(0,1) 2317   0.5118

```



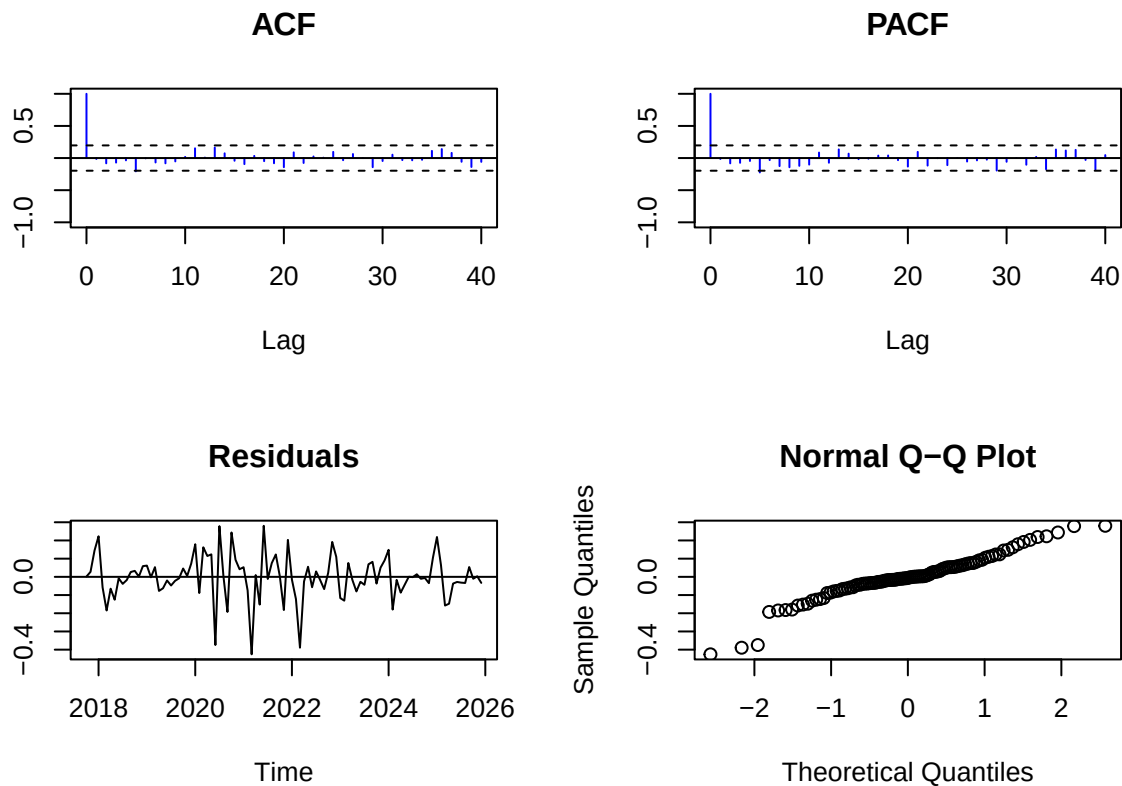
```
test(fit_clean$residuals)
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)    15.61   0.7405
## McLeod-Li Q    Q ~ chisq(20)    58.39    0 *
## Turning points T  (T-64.7)/4.2 ~ N(0,1)    63   0.6884
## Diff signs S     (S-49)/2.9 ~ N(0,1)    53   0.1659
## Rank P          (P-2425.5)/165.4 ~ N(0,1) 2283  0.3889
```



```
test(fit_log_clean$residuals)
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q   Q ~ chisq(20)    18.99   0.5226
## McLeod-Li Q   Q ~ chisq(20)    53.07   1e-04 *
## Turning points T (T-64.7)/4.2 ~ N(0,1)    65   0.9361
## Diff signs S   (S-49)/2.9 ~ N(0,1)    52   0.2987
## Rank P         (P-2425.5)/165.4 ~ N(0,1) 2333   0.576
```

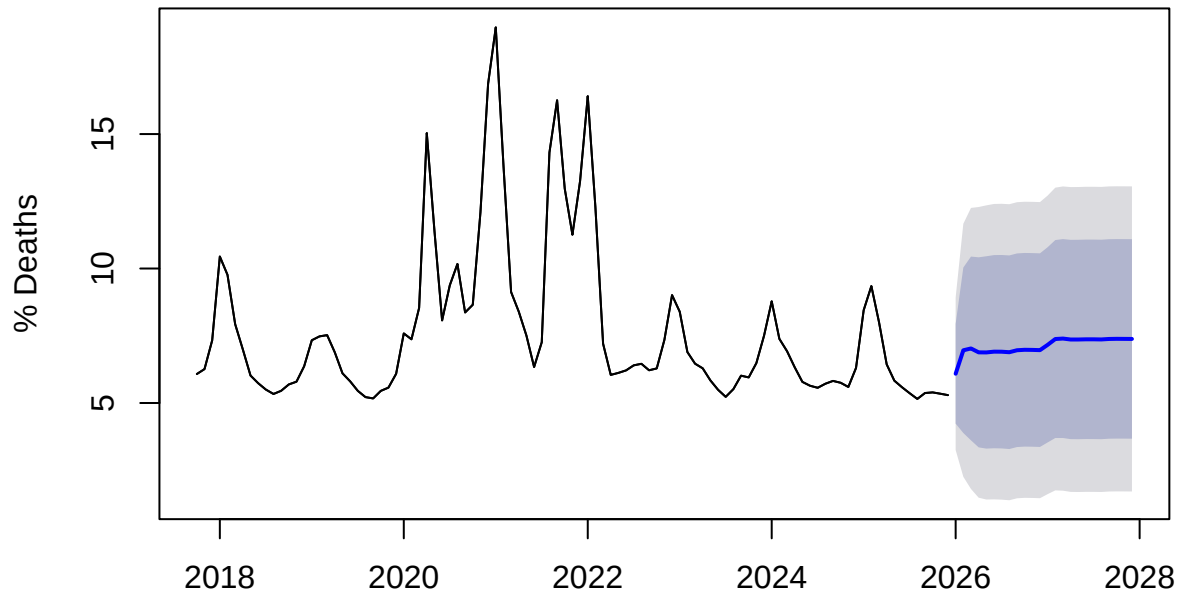


The log-transformed model on the cleaned series became the final forecasting model. Below are all three forecasts for comparison, then the chosen 24-month forecast with its values tabulated.

```
# Generate Forecasts
fc_raw <- forecast::forecast(fit_raw, h=24)
fc_clean <- forecast::forecast(fit_clean, h=24)
fc_log_clean <- forecast::forecast(fit_log_clean, h=24, biasadj=TRUE)

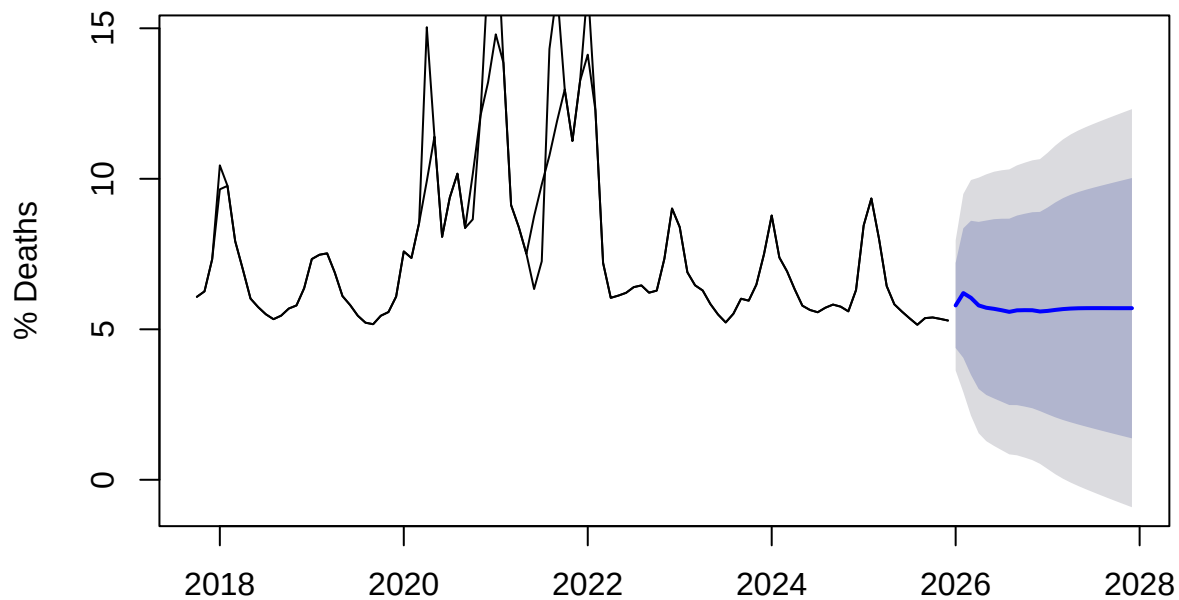
plot(fc_raw, main="Forecast: Raw Model (No Log Transform)", ylab="% Deaths", fcol="blue")
lines(ts_raw, col="black")
```

Forecast: Raw Model (No Log Transform)



```
# PLOT 1: Cleaned Data Model
plot(fc_clean, main="Forecast: Cleaned Model (No Log Transform)", ylab="% Deaths", fcol="blue")
lines(ts_raw, col="black")
```

Forecast: Cleaned Model (No Log Transform)

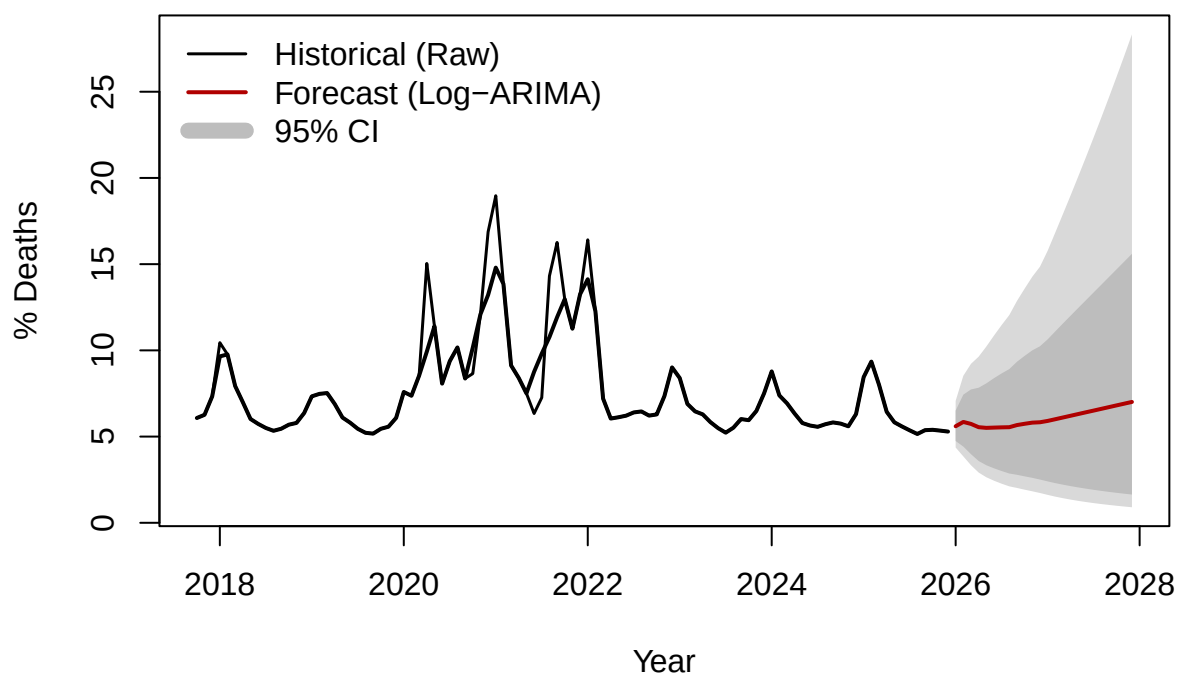


```
plot(fc_log_clean,
     main="24-Month Forecast: US P&I Mortality (Log-Transformed+ Cleaned)",
     ylab="% Deaths",
     xlab="Year",
     fcol="#b00000",          # Dark Red
     flty=1, lwd=2,
     shadecols=c("#d9d9d9", "#bdbdbd")
)          # Focus on realistic range

# Overlay Historical Data
lines(ts_raw, col="black", lwd=1.5)

legend("topleft",
      legend=c("Historical (Raw)", "Forecast (Log-ARIMA)", "95% CI"),
      col=c("black", "#b00000", "#bdbdbd"),
      lty=c(1, 1, 1), lwd=c(1.5, 2, 8), bty="n")
```

24-Month Forecast: US P&I Mortality (Log-Transformed+ Cleaned)



```
# Extract forecast values into a readable dataframe
fc_values <- data.frame(
  Date = seq(from = max(monthly_data$MonthDate) %m+% months(1), by = "month", length.out = 24),
  Forecast = round(fc_log_clean$mean, 2),
  Lower_95 = round(fc_log_clean$lower[, "95%"], 2),
  Upper_95 = round(fc_log_clean$upper[, "95%"], 2)
)

kable(fc_values, caption = "24-Month Forecast Values (Log-ARIMA)")
```

Table 2: 24-Month Forecast Values (Log-ARIMA)

Date	Forecast	Lower_95	Upper_95
2026-01-01	5.60	4.37	7.07
2026-02-01	5.85	3.85	8.53
2026-03-01	5.73	3.34	9.22
2026-04-01	5.54	2.91	9.63
2026-05-01	5.51	2.64	10.22
2026-06-01	5.52	2.44	10.86
2026-07-01	5.54	2.27	11.47
2026-08-01	5.54	2.11	12.05
2026-09-01	5.67	2.02	12.87
2026-10-01	5.74	1.92	13.58
2026-11-01	5.81	1.83	14.27

Date	Forecast	Lower_95	Upper_95
2026-12-01	5.83	1.73	14.85
2027-01-01	5.91	1.62	15.78
2027-02-01	6.01	1.52	16.84
2027-03-01	6.11	1.43	17.91
2027-04-01	6.21	1.35	18.99
2027-05-01	6.31	1.27	20.09
2027-06-01	6.41	1.20	21.21
2027-07-01	6.51	1.14	22.34
2027-08-01	6.61	1.09	23.50
2027-09-01	6.71	1.04	24.67
2027-10-01	6.81	0.99	25.87
2027-11-01	6.91	0.94	27.09
2027-12-01	7.01	0.90	28.33

Hybrid ARIMA plus GARCH

The ARIMA mean forecast assumes constant variance, but mortality spikes cluster. So I fit a GARCH(1,1) with Student-t errors on the raw ARIMA residuals, deliberately using the raw fit's residuals to keep the real spikes in the volatility estimate. The conditional volatility plot shows when uncertainty was highest.

```
library(rugarch)
# 1. Extract Residuals from the best ARIMA fit
best_residuals <- residuals(fit_raw) # Using Raw ARIMA residuals to capture the real spikes

# 2. Define GARCH(1,1) Specification
garch_spec <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  mean.model = list(armaOrder = c(0, 0), include.mean = FALSE),
  distribution.model = "std"
)

# 3. Fit GARCH
garch_fit <- ugarchfit(spec = garch_spec, data = best_residuals)

# 4. Diagnostics
print(garch_fit)
```

```
##
## *-----*
## *          GARCH Model Fit          *
## *-----*
##
## Conditional Variance Dynamics
## -----
## GARCH Model   : sGARCH(1,1)
## Mean Model    : ARFIMA(0,0,0)
## Distribution   : std
##
## Optimal Parameters
## -----
##      Estimate Std. Error  t value Pr(>|t|)
```

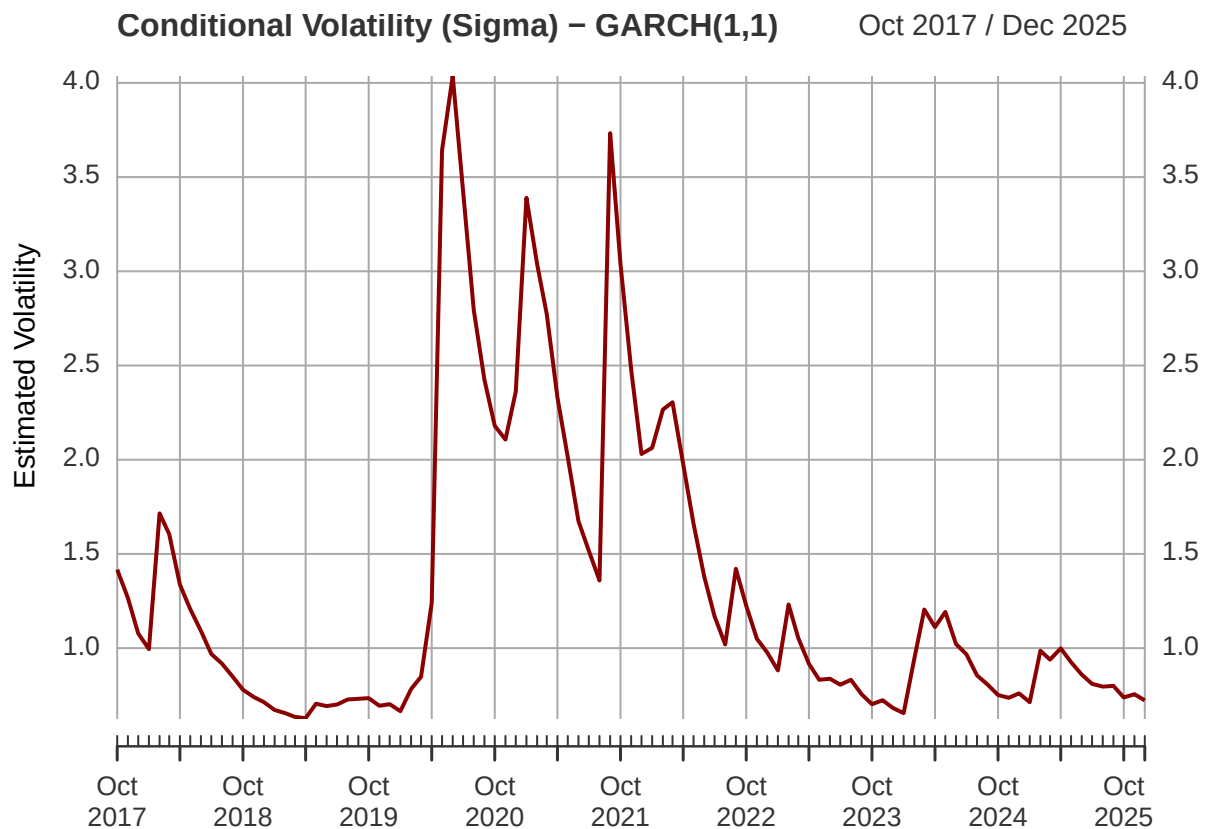
```

## omega      0.12151      0.10091      1.2041 0.228543
## alpha1     0.35169      0.24499      1.4355 0.151140
## beta1      0.64731      0.14834      4.3637 0.000013
## shape      3.19291      0.90405      3.5318 0.000413
##
## Robust Standard Errors:
##      Estimate Std. Error t value Pr(>|t|)
## omega      0.12151      0.11448      1.0614 0.288507
## alpha1     0.35169      0.32637      1.0776 0.281214
## beta1      0.64731      0.21312      3.0373 0.002387
## shape      3.19291      0.78369      4.0742 0.000046
##
## LogLikelihood : -139.523
##
## Information Criteria
## -----
##
## Akaike          2.8995
## Bayes           3.0043
## Shibata         2.8964
## Hannan-Quinn    2.9419
##
## Weighted Ljung-Box Test on Standardized Residuals
## -----
##
##              statistic p-value
## Lag[1]              1.215 0.2704
## Lag[2*(p+q)+(p+q)-1] [2] 1.220 0.4321
## Lag[4*(p+q)+(p+q)-1] [5] 1.623 0.7095
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## -----
##
##              statistic p-value
## Lag[1]              0.4892 0.4843
## Lag[2*(p+q)+(p+q)-1] [5] 0.9323 0.8748
## Lag[4*(p+q)+(p+q)-1] [9] 1.4348 0.9607
## d.o.f=2
##
## Weighted ARCH LM Tests
## -----
##
##      Statistic Shape Scale P-Value
## ARCH Lag[3]    0.2877 0.500 2.000 0.5917
## ARCH Lag[5]    0.5483 1.440 1.667 0.8693
## ARCH Lag[7]    0.6170 2.315 1.543 0.9666
##
## Nyblom stability test
## -----
## Joint Statistic: 0.6037
## Individual Statistics:
## omega 0.11355
## alpha1 0.14028
## beta1 0.08156
## shape 0.09312

```

```
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:      1.07 1.24 1.6
## Individual Statistic:  0.35 0.47 0.75
##
## Sign Bias Test
## -----
##           t-value   prob sig
## Sign Bias      2.3810 0.01928 **
## Negative Sign Bias 0.1198 0.90486
## Positive Sign Bias 0.1546 0.87750
## Joint Effect      9.8625 0.01977 **
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## -----
##   group statistic p-value(g-1)
## 1    20      45.85   0.0005207
## 2    30      47.97   0.0148046
## 3    40      58.17   0.0247384
## 4    50      79.28   0.0039810
##
##
## Elapsed time : 0.1352711
```

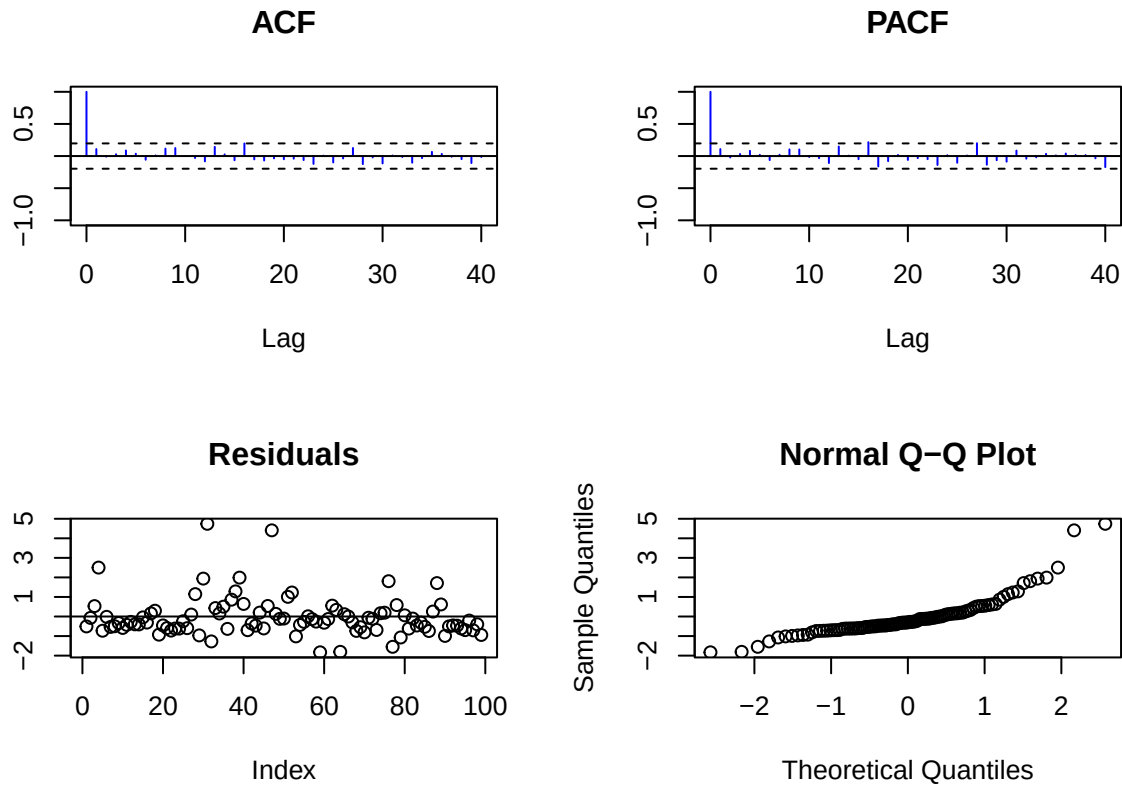
```
# Plot Conditional Volatility (Sigma)
volatility <- sigma(garch_fit)
plot(volatility, type="l", col="darkred",
     main="Conditional Volatility (Sigma) - GARCH(1,1)",
     ylab="Estimated Volatility", xlab="Time")
grid()
```



```
# Extract Standardized Residuals
std_resid <- residuals(garch_fit, standardize=TRUE)

# Diagnostic Tests (Ljung-Box, etc.)
test(as.numeric(std_resid))
```

```
## Null hypothesis: Residuals are iid noise.
## Test          Distribution Statistic  p-value
## Ljung-Box Q    Q ~ chisq(20)      15.72   0.734
## McLeod-Li Q    Q ~ chisq(20)      24.31   0.2289
## Turning points T (T-64.7)/4.2 ~ N(0,1) 63    0.6884
## Diff signs S   (S-49)/2.9 ~ N(0,1)  52    0.2987
## Rank P         (P-2425.5)/165.4 ~ N(0,1) 2240   0.262
```



Results

The final hybrid forecast combines the ARIMA mean path with GARCH-driven 95% intervals: the interval width at each horizon comes from the forecasted conditional volatility rather than a constant variance assumption.

8. Final Hybrid Forecasting

```
forecast_horizon <- 24
```

1. ARIMA Forecast (Mean)

```
fc_arima <- forecast::forecast(fit_raw, h=forecast_horizon)
```

2. GARCH Forecast (Variance/Sigma)

```
garch_fc <- ugarchforecast(garch_fit, n.ahead = forecast_horizon)
```

```
sigma_forecast <- sigma(garch_fc)
```

3. Create Hybrid Dataframe

```
start_date <- max(monthly_data$MonthDate)
```

```
future_dates <- seq(from = start_date, by = "month", length.out = forecast_horizon + 1)[-1]
```

```
hybrid_df <- data.frame(
  Date = future_dates,
  Mean_Forecast = as.numeric(fc_arima$mean),
```

```

Sigma_GARCH = as.numeric(sigma_forecast)
)

# Calculate 95% Confidence Intervals
hybrid_df$Lower_95 <- hybrid_df$Mean_Forecast - (1.96 * hybrid_df$Sigma_GARCH)
hybrid_df$Upper_95 <- hybrid_df$Mean_Forecast + (1.96 * hybrid_df$Sigma_GARCH)

# 4. Final Plot

library(lubridate)
forecast_time_axis <- decimal_date(hybrid_df$Date)

# 2. Define Plot Limits
start_plot <- 2018
end_plot <- max(time(ts_raw)) + (24/12)

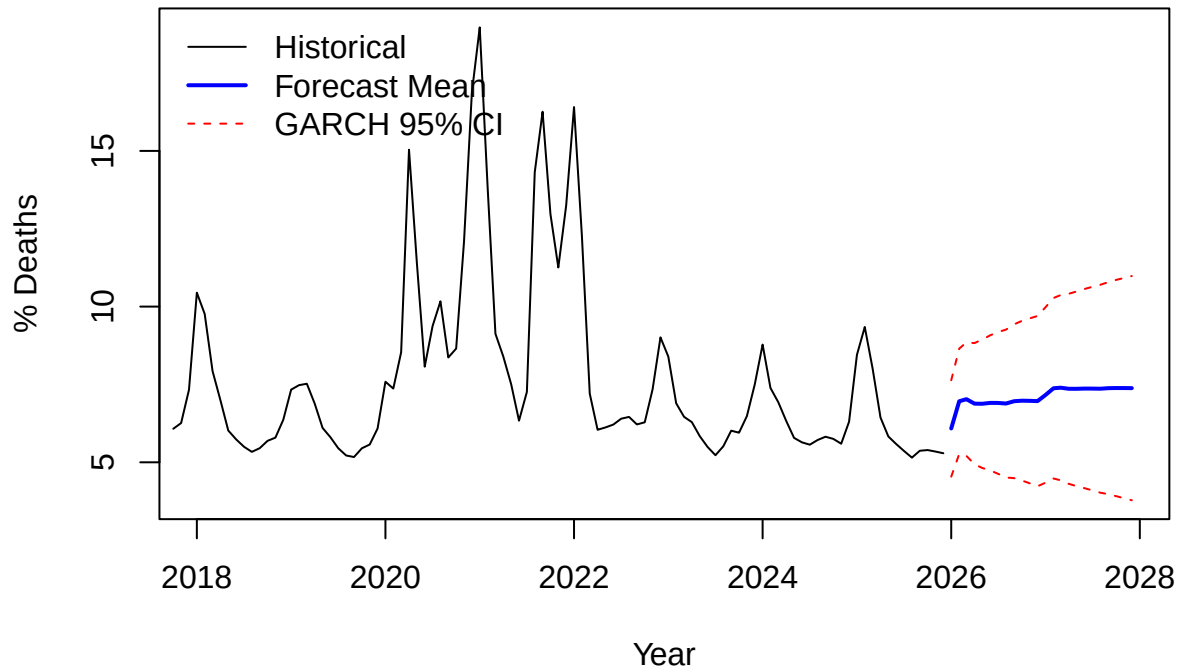
# 3. Plot
plot(ts_raw,
     main="24-Month Forecast: Hybrid ARIMA + GARCH(1,1)",
     ylab="% Deaths", xlab="Year",
     xlim=c(start_plot, end_plot),
     ylim=c(min(hybrid_df$Lower_95), max(ts_raw)),
     col="black", lwd=1)

# 4. Add Lines using the converted time axis
lines(forecast_time_axis, hybrid_df$Mean_Forecast, col="blue", lwd=2)
lines(forecast_time_axis, hybrid_df$Upper_95, col="red", lty=2)
lines(forecast_time_axis, hybrid_df$Lower_95, col="red", lty=2)

legend("topleft",
     legend=c("Historical", "Forecast Mean", "GARCH 95% CI"),
     col=c("black", "blue", "red"),
     lty=c(1, 1, 2), lwd=c(1, 2, 1), bty="n")

```

24-Month Forecast: Hybrid ARIMA + GARCH(1,1)



Takeaway

The cleaning step was the difference between a model that behaves and one that does not: comma-formatted counts and reporting artifacts had to be handled before any ARIMA would fit sensibly. On the cleaned series, the log-transformed ARIMA gave the final 24-month mean forecast, and layering GARCH(1,1) volatility on the residuals produced honest 95% intervals that widen where history was volatile. The hybrid keeps the seasonal structure from ARIMA while admitting that uncertainty itself clusters in time.